機能管理ダイアログUI設計書

メタデータ

title: 機能管理ダイアログUI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-02-02
author: Development Team
tags: [system-admin, role-management, dialog, function-permission]

関連ドキュメント

- ロール管理画面
- ロール追加ダイアログ
- ロール編集ダイアログ
- ロール管理API仕様
- UIポリシー

---

1. 概要

1.1. 目的

システム管理者がロールに対して機能（メニュー）とその権限（書込可・読取専用）を設定するためのモーダルダイアログです。機能の割り当て、権限の設定、子機能の依存関係管理を行います。

1.2. 表示条件

呼び出し元:

1. ロール管理画面のテーブル内「機能名」カラムをクリック（directモード）
2. ロール追加ダイアログの「機能名」フィールドをクリック（temporaryモード）
3. ロール編集ダイアログの「機能名」フィールドをクリック（directモード）

必要な権限:

- System Admin権限

1.3. 動作モード

モード	説明	呼び出し元	DB反映タイミング
direct	DB直接更新モード	ロール管理画面、ロール編集ダイアログ	「機能を更新」ボタンクリック時
temporary	メモリ一時保存モード	ロール追加ダイアログ	親ダイアログの「保存」ボタンクリック時

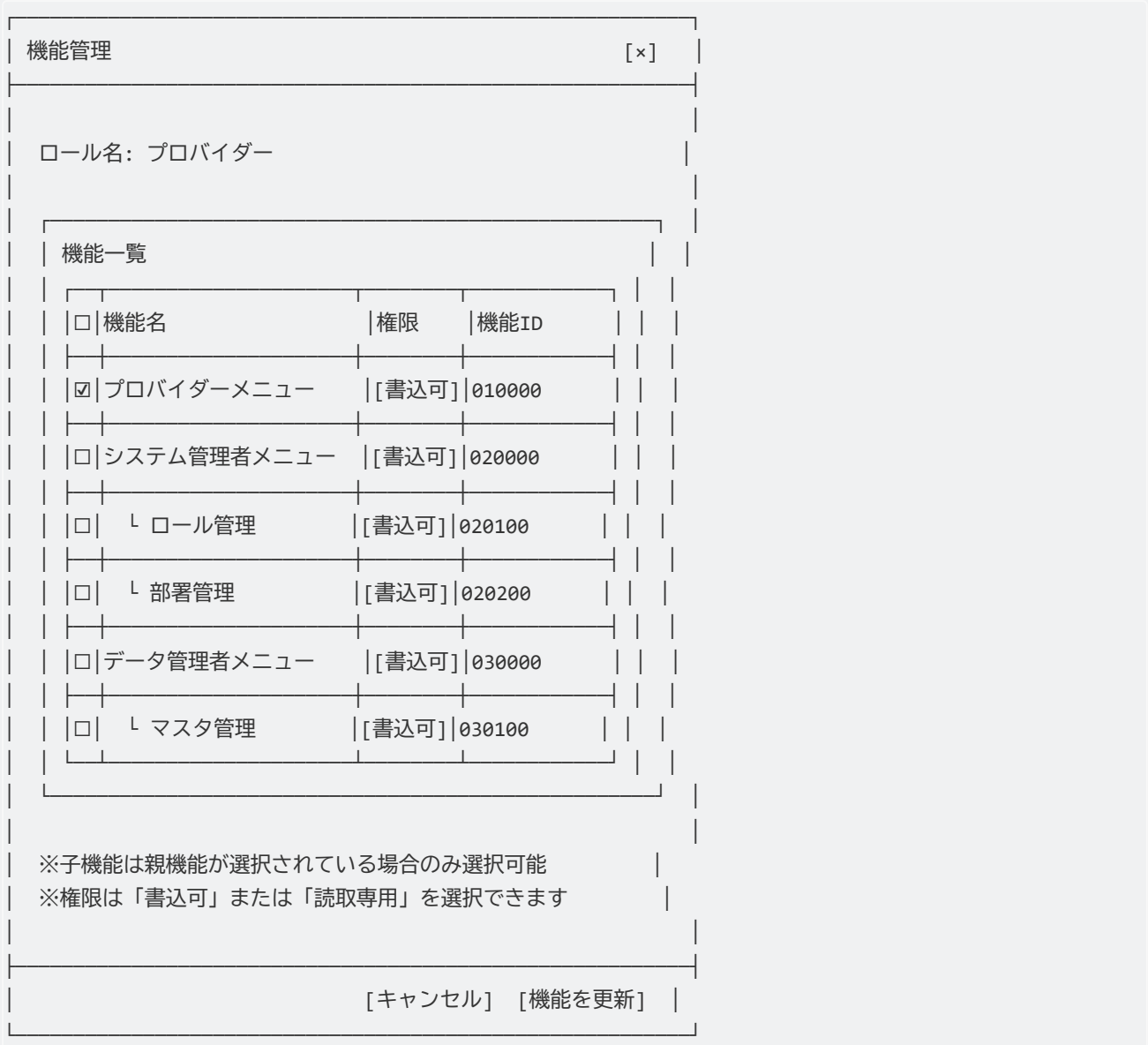
1.4. 画面イメージ



---

2. ダイアログ構成

2.1. レイアウト



2.2. モーダル仕様

コンポーネント: `<Modal>`

仕様:

項目	値
タイトル	「機能管理」
幅	800px
位置	画面中央
最大高さ	600px

マスク	背景を暗くする
キーボードESC	閉じる（確認なし）
マスククリック	閉じない

---

3. ヘッダー情報

3.1. ロール名表示

表示内容: `ロール名：{roleName}`

例: `ロール名：プロバイダー`

スタイル:

```
<Typography.Text strong style={{ fontSize: 16, marginBottom: 16 }}>
  ロール名: {currentMaster?.roleName}
</Typography.Text>
```

---

4. 機能一覧テーブル

4.1. テーブル仕様

コンポーネント: `<Table>`

仕様:

項目	値
サイズ	`small`
行選択	チェックボックスあり
ページング	なし（全件表示）
スクロール	縦スクロール有効（y: 400）

4.2. カラム定義

カラム名	データ項目	幅	配置	備考
（チェックボックス）	-	50px	center	機能選択用
機能名	`functionName`	300px	left	階層表示（親/子）
権限	`permission`	150px	center	ラジオボタン切替
機能ID	`functionId`	100px	left	識別用

4.3. 機能名カラムの階層表示

親機能の表示:

```
プロバイダーメニュー
システム管理者メニュー
データ管理者メニュー
実験管理者メニュー
実験作業者メニュー
```

子機能の表示:

```
└ ロール管理
└ 部署管理
└ マスタ管理
```

実装例:

```
render: (text, record) => {
  const isChild = record.parentFunctionId !== null;
  return (
    <span style={{ paddingLeft: isChild ? '24px' : '0' }}>
      {isChild && '└ '}
      {text}
    </span>
  );
}
```

4.4. 権限カラム（ラジオボタン）

コンポーネント: `<Radio.Group>`

選択肢:

- **書込可**: データの作成・更新・削除が可能
- **読取専用**: データの閲覧のみ可能

初期状態:

- チェックボックスが選択されている場合のみ活性化
- 未選択の場合は非活性（グレーアウト）
- デフォルト値: 書込可

実装例:

```
<Radio.Group
  value={record.permission}
  onChange={(e) => handlePermissionChange(record.functionId, e.target.value)}
  disabled={!selectedFunctionIds.includes(record.functionId)}
>
  <Radio value="writable">書込可</Radio>
  <Radio value="readonly">読取専用</Radio>
</Radio.Group>
```

4.5. 子機能の依存関係制御

ルール:

1. 親機能が選択されていない場合、子機能は選択不可（チェックボックス非活性）
2. 親機能のチェックを外すと、子機能のチェックも自動的に外れる
3. 親機能が読取専用の場合、子機能も読取専用になる（変更不可）
4. 親機能が書込可の場合、子機能は書込可または読取専用を選択できる

実装例:

```
// チェックボックスの非活性制御
const isCheckboxDisabled = (record: FunctionRecord) => {
  if (!record.parentFunctionId) return false; // 親機能は常に選択可能
  return !selectedFunctionIds.includes(record.parentFunctionId); // 親が未選択なら非活性
};

// 親機能チェック解除時の処理
const handleCheckboxChange = (functionId: string, checked: boolean) => {
  if (!checked) {
    // 親機能のチェックを外す場合、子機能も外す
    const childFunctions = functions.filter(f => f.parentFunctionId === functionId);
    const childIds = childFunctions.map(f => f.functionId);
    setSelectedFunctionIds(prev => prev.filter(id => id !== functionId &&
!childIds.includes(id)));
  } else {
    setSelectedFunctionIds(prev => [...prev, functionId]);
  }
};

// 権限変更時の子機能への伝播
const handlePermissionChange = (functionId: string, permission: 'writable' | 'readonly') => {
  // 親機能の場合、子機能の権限も変更
  const childFunctions = functions.filter(f => f.parentFunctionId === functionId);
  const updates = { [functionId]: permission };

  if (permission === 'readonly') {
    // 親が読取専用になる場合、子も読取専用にする
    childFunctions.forEach(child => {
      updates[child.functionId] = 'readonly';
    });
  }

  setPermissions(prev => ({ ...prev, ...updates }));
};
```

---

5. アクションボタン

5.1. 配置

位置: モーダルフッター（右寄せ）

レイアウト:

```
<Space>
  <Button onClick={handleCancel}>キャンセル</Button>
  <Button type="primary" onClick={handleUpdate}>機能を更新</Button>
</Space>
```

5.2. キャンセルボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「キャンセル」
タイプ	`default`

動作:

1. モーダルを閉じる
2. 変更内容を破棄
3. 確認なし

5.3. 機能を更新ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「機能を更新」
タイプ	`primary`
ローディング	送信中は `true`

動作（directモード）:

1. ローディング状態に変更
2. API呼び出し（`updateRolePermissions`）

3. 成功時:
    - モーダルを閉じる
    - 親画面のデータを更新
    - 成功メッセージ表示: `"機能を更新しました"`
4. 失敗時:
    - エラーメッセージ表示
    - モーダルは開いたまま

動作（temporaryモード）:

1. 選択された機能と権限をメモリに一時保存
2. モーダルを閉じる
3. 親ダイアログ（ロール追加）の機能名フィールドを更新
4. 初期画面機能名フィールドを活性化
5. DBには保存しない（親ダイアログの「保存」ボタンで保存）

実装例（directモード）:

```
const handleUpdate = async () => {
  try {
    setLoading(true);

    // 機能名と権限情報を生成
    const functionNameDisplay = generateFunctionNameDisplay(
      selectedFunctionIds,
      permissions
    );

    const functionWritePermissions = generateFunctionWritePermissions(
      selectedFunctionIds,
      permissions
    );

    // API呼び出し
    await updateRolePermissions(
      currentMaster.role,
      functionNameDisplay,
      functionWritePermissions
    );
```

```
    message.success('機能を更新しました');

    // 親画面のコールバックを実行
    onUpdate?.();

    onClose();
  } catch (error) {
    console.error('エラー:', error);
    message.error('機能の更新に失敗しました');
  } finally {
    setLoading(false);
  }
};
```

実装例（temporaryモード）：

```
const handleUpdate = () => {
  // メモリに一時保存
  const functionNameDisplay = generateFunctionNameDisplay(
    selectedFunctionIds,
    permissions
  );

  const functionWritePermissions = generateFunctionWritePermissions(
    selectedFunctionIds,
    permissions
  );

  onTemporarySave?.({
    functionName: functionNameDisplay,
    functionWritePermissions: functionWritePermissions,
    functionIds: selectedFunctionIds,
    writableFunctionIds: Object.keys(permissions).filter(
      id => permissions[id] === 'writable'
    ),
  });

  onClose();
};
```

---

6. 機能名・権限の生成ロジック

6.1. 機能名表示の生成

フォーマット: `Ⓦ{機能名} / Ⓡ{機能名} / ...`

権限アイコン:

- Ⓦ: 書込可（writable）
- Ⓡ: 読取専用（readonly）

生成ルール:

1. 選択された機能のみを表示
2. 権限に応じてアイコンを付与
3. 複数機能は `/` で区切る
4. 表示順序: 機能ID順（親機能優先）

実装例:

```
const generateFunctionNameDisplay = (
  selectedIds: string[],
  permissions: Record<string, 'writable' | 'readonly'>
): string => {
  const selectedFunctions = functions
    .filter(f => selectedIds.includes(f.functionId))
    .sort((a, b) => a.functionId.localeCompare(b.functionId));

  return selectedFunctions
    .map(func => {
      const icon = permissions[func.functionId] === 'writable' ? 'Ⓦ' : 'Ⓡ';
      return `${icon}${func.functionName}`;
    })
    .join(' / ');
};
```

例:

```
入力:
  - プロバイダーメニュー（書込可）
  - システム管理者メニュー（読取専用）
```

```
出力:
  "Ⓦプロバイダーメニュー / Ⓡシステム管理者メニュー"
```

6.2. 書込可能機能名の生成

フォーマット: `{機能名}, {機能名}, ...`

生成ルール:

1. 書込可の機能のみを抽出
2. カンマ+スペースで区切る
3. 権限アイコンは付けない

実装例:

```
const generateFunctionWritePermissions = (
  selectedIds: string[],
  permissions: Record<string, 'writable' | 'readonly'>
): string => {
  const writableFunctions = functions
    .filter(f =>
      selectedIds.includes(f.functionId) &&
      permissions[f.functionId] === 'writable'
    )
    .sort((a, b) => a.functionId.localeCompare(b.functionId));

  return writableFunctions
    .map(func => func.functionName)
    .join(', ');
};
```

例:

```
入力:
  - プロバイダーメニュー（書込可）
  - システム管理者メニュー（読取専用）

出力:
  "プロバイダーメニュー"
```

---

7. API仕様

7.1. ロール権限更新（directモード）

エンドポイント: `PUT /api/v1/systemManagement/role/:roleId/permissions`

リクエストヘッダー:

```
Content-Type: application/json
Authorization: Bearer {session_token}
```

リクエストボディ:

```
{
  "functionName": "Ⓦプロバイダーメニュー / Ⓡシステム管理者メニュー",
  "functionWritePermissions": "プロバイダーメニュー"
}
```

レスポンス（成功時）:

```
{
  "success": true,
  "meta": {
    "timestamp": "2026-02-02T09:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```

7.2. ロール詳細取得（directモード用）

エンドポイント: `GET /api/v1/systemManagement/role/:roleId`

リクエストヘッダー:

```
Authorization: Bearer {session_token}
```

レスポンス（成功時）:

```
{
  "success": true,
  "data": {
    "roleId": "provider",
    "roleName": "プロバイダー",
    "functionName": "Ⓦプロバイダーメニュー",
    "functionWritePermissions": "プロバイダーメニュー",
    "functionIds": ["010000"],
    "writableFunctionIds": ["010000"],
    "isActive": true
  },
  "meta": {
    "timestamp": "2026-02-02T09:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```

7.3. 機能マスター一覧取得

エンドポイント: `GET /api/v1/systemManagement/functions`

リクエストヘッダー:

```
Authorization: Bearer {session_token}
```

レスポンス（成功時）:

```
{
  "success": true,
  "data": [
    {
      "functionId": "010000",
      "functionName": "プロバイダーメニュー",
      "parentFunctionId": null
    },
```

```
    {
      "functionId": "020000",
      "functionName": "システム管理者メニュー",
      "parentFunctionId": null
    },
    {
      "functionId": "020100",
      "functionName": "ロール管理",
      "parentFunctionId": "020000"
    },
    {
      "functionId": "020200",
      "functionName": "部署管理",
      "parentFunctionId": "020000"
    }
  ],
  "meta": {
    "timestamp": "2026-02-02T09:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```

---

8. エラーハンドリング

8.1. エラーメッセージ一覧

エラーコード	メッセージ	表示場所
`VALIDATION_ERROR`	「機能を1つ以上選択してください」	トースト
`RESOURCE_NOT_FOUND`	「指定されたロールが見つかりません」	トースト
`SYSTEM_ERROR`	「機能の更新に失敗しました」	トースト

8.2. エラー表示方法

トーストメッセージ:

```
message.error('機能の更新に失敗しました');
```

---

9. ユーザーフロー

9.1. 正常フロー（directモード）

```
[ロール管理画面] または [ロール編集ダイアログ]
    ↓ 機能名フィールドクリック
[機能管理ダイアログ] 表示（directモード）
    DBから最新のロール情報を取得
    現在の機能選択状態を表示
    ↓ 機能を選択/解除
    ↓ 権限を変更（書込可 ⇔ 読取専用）
    ↓ 「機能を更新」ボタンクリック
API呼び出し: updateRolePermissions()
    ↓ 成功（DBに即座に反映）
親画面に戻る
    機能名フィールドが更新される
    成功メッセージ表示: "機能を更新しました"
```

9.2. 正常フロー（temporaryモード）

```
[ロール追加ダイアログ]
    ↓ 機能名フィールドクリック
[機能管理ダイアログ] 表示（temporaryモード）
    空の状態で表示
    ↓ 機能を選択
    ↓ 権限を設定（書込可 / 読取専用）
    ↓ 「機能を更新」ボタンクリック
メモリに一時保存（DBには未保存）
[ロール追加ダイアログ] に戻る
    機能名フィールドが更新される
    初期画面機能名フィールドが活性化される
    ↓ その他の項目を入力
    ↓ 「保存」ボタンクリック
API呼び出し: createRole() + updateRolePermissions()
    ↓ 成功（DBに保存）
[ロール管理画面]
    データ再読み込み
```

9.3. 子機能依存関係のフロー

```
[機能管理ダイアログ]
    ↓ 親機能（例：システム管理者メニュー）を選択
    子機能（ロール管理、部署管理）が選択可能になる
    ↓ 子機能を選択
    ↓ 親機能を読取専用に変更
    子機能も自動的に読取専用になる
    ↓ 親機能のチェックを外す
    子機能のチェックも自動的に外れる
```

---

10. モードごとの動作比較

項目	directモード	temporaryモード
呼び出し元	ロール管理画面、ロール編集ダイアログ	ロール追加ダイアログ
初期表示	DBから現在の設定を取得	空の状態または一時保存データ
更新タイミング	「機能を更新」ボタンクリック時	親ダイアログの「保存」ボタンクリック時
API呼び出し	あり（即座にDB更新）	なし（メモリに保存）
キャンセル時	変更を破棄	一時保存データを破棄
成功メッセージ	「機能を更新しました」	なし

---

11. アクセシビリティ

11.1. キーボード操作

Tab順序:

1. 機能一覧テーブル（各チェックボックス）
2. 権限ラジオボタン（活性化されている行のみ）
3. キャンセルボタン
4. 機能を更新ボタン

ショートカットキー:

- Space: チェックボックスの切り替え

- Enter: 機能を更新ボタンのクリック
- Escape: モーダルを閉じる

11.2. ARIA属性

```
<Modal
  title="機能管理"
  open={isVisible}
  onCancel={handleCancel}
  aria-labelledby="function-management-dialog-title"
  aria-describedby="function-management-dialog-description"
>
  <Table
    aria-label="機能一覧"
    rowKey="functionId"
    {...otherProps}
  />
</Modal>
```

---

12. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-02-02	初版作成	Development Team